



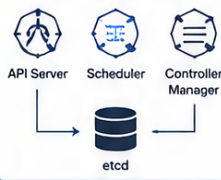
HOW KUBERNETES WORKS (END-TO-END)

From Your Request to a Running, Healthy Application

1 Cluster Setup

A Kubernetes cluster has Control Plane components that manage the cluster and Worker Nodes where your applications run.

CONTROL PLANE



WORKER NODES



2 Define Desired State

You define your application using YAML manifests.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: app
          image: my-app:1.0
```



3 Submit Request

You run the command to apply your manifest.

```
$ kubectl apply -f app.yaml
```



- Authentication (Who are you?)
- Authorization (What can you do?)
- Admission Controllers (Validate the request)

4 API Server Validates & Stores State

- Request is validated against schema
 - Stored in etcd (cluster's key-value database)
- This becomes the **Source of Truth**



Source of Truth

5 Controllers Watch for Changes

Controllers continuously watch the API Server for changes in desired state.

Examples:

- Deployment Controller
- ReplicaSet Controller
- StatefulSet Controller
- Node Controller
- Endpoint Controller
- Service Controller



6 Reconciliation Loop (Actual vs Desired State)

Controllers compare the Desired State (from etcd) with the Actual State (from the cluster). If there is a difference, they take action to make the actual state match the desired state.

Example Flow: Deployment → ReplicaSet → Pods



7 Scheduler Assigns Pods to Nodes

Scheduler watches for unscheduled Pods and selects the best node based on:

- CPU / Memory availability
- Node Affinity / Anti-Affinity
- Taints & Tolerations
- Topology rules
- Other scheduling policies

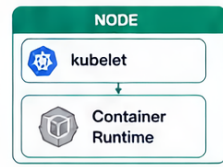


SCHEDULER

8 Pod Assigned → Kubelet Takes Over

The Pod spec is sent to the selected node. Kubelet on the node:

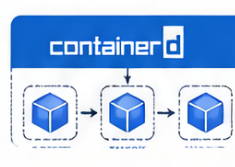
- Receives the Pod spec
- Ensures containers should be running
- Talks to the container runtime



9 Container Runtime Runs Containers

Container runtime (containerd / CRI-O):

- Pulls image from registry
- Creates containers
- Starts containers inside the Pod



10 Networking (CNI Plugin)

CNI plugin sets up network for the Pod.

- Assigns Pod IP
- Configures routing
- Enables communication within the cluster

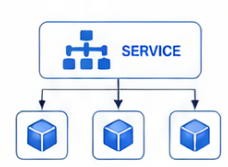


Pod IP
10.244.1.5

11 Service Networking (kube-proxy / eBPF)

kube-proxy (or eBPF data plane) sets up rules to:

- Load balance traffic
- Route to healthy Pods
- Provide stable virtual IP for Services



12 DNS & Service Discovery

CoreDNS provides DNS resolution for Services and Pods.



CoreDNS

```
service-name.namespace.svc.cluster.local
pod-ip.namespace.pod.cluster.local
```

Enables seamless communication between applications.

13 Health Checks & Status Updates

Kubelet performs health checks and reports status back to API Server.

- Liveness Probe
- Readiness Probe
- Startup Probe



Status is stored in etcd and visible via `kubectl get pods`

14 Self-Healing

If something goes wrong:

- Pod crashes → Restarted
- Node fails → Pod rescheduled
- Health check fails → Removed from service



Kubernetes automatically detects and heals.

15 Scaling

Scale your application as needed.

- Manual Scaling (replicas)
- Horizontal Pod Autoscaler (HPA)
- Vertical Pod Autoscaler (VPA)
- Cluster Autoscaler (adds/removes nodes)



16 Rolling Updates & Rollbacks

Kubernetes updates your application with zero downtime.

- Rolling updates (new version gradually)
- Monitors health
- Automatic rollback if failure detected



17 Continuous Reconciliation Loop

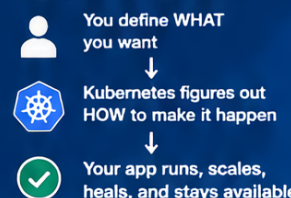
Kubernetes never stops!

It constantly watches the cluster, detects drift, and reconciles the actual state to match the desired state.



THE BIG PICTURE

You declare the desired state. Kubernetes handles the rest.



Key Takeaway

Kubernetes is a declarative, self-healing, and automated system that manages containers at scale. You focus on your application. Kubernetes takes care of the complexity.